Linear Transformations Between Steering Vectors in Large Language Models

Soumyajit Karmakar
UC San Diego
skarmakar@ucsd.edu

Neil Mallinar
UC San Diego
nmallinar@gmail.com

Mikhail Belkin
UC San Diego
mbelkin@ucsd.edu

Abstract

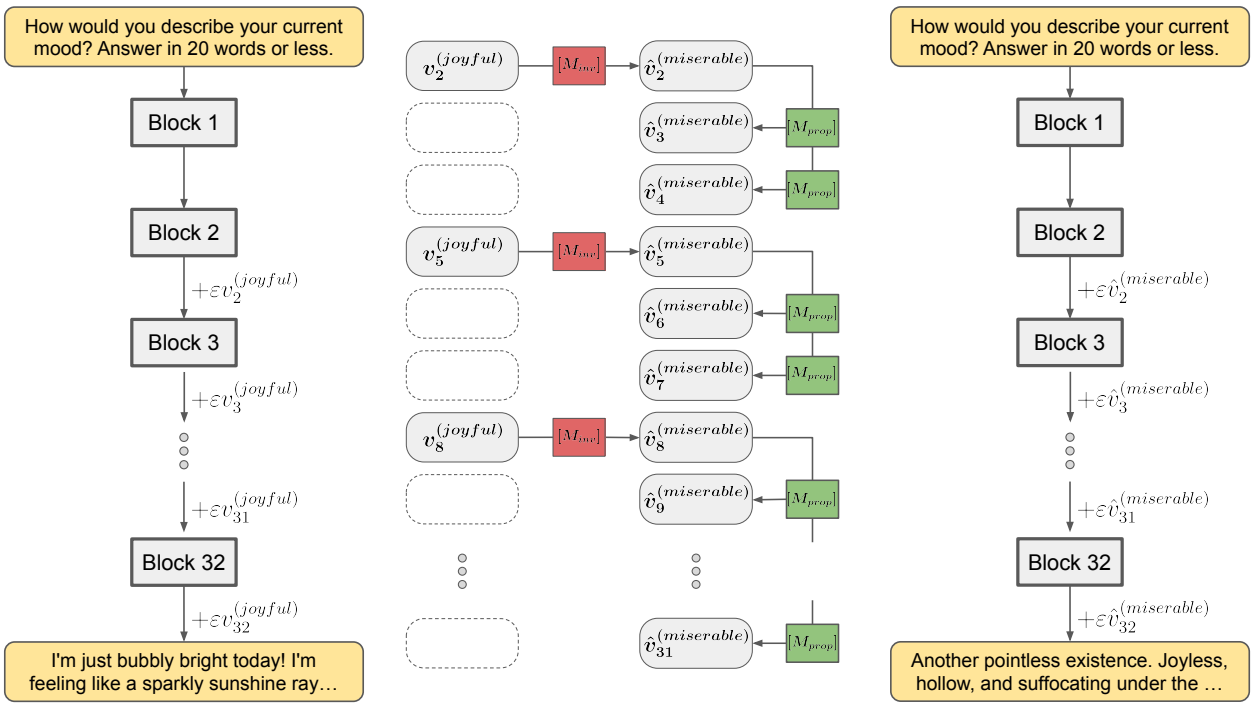
The behavior of Large Language Models (LLMs) can be altered at inference time by adding steering vectors to their internal activations. Existing methods typically extract these vectors independently for each target concept and model layer. In this work, we study whether related steering vectors are instead connected by shared linear transformations across concepts, tasks, and model depth. We find that such transformations, learned on a subset of concepts, generalize to held-out concepts. They can map concepts to their semantic opposites, propagate steering vectors across model layers, and modify the source or target language represented by translation steering vectors. Furthermore, a single inversion or propagation matrix shared by all layers performs competitively with layer-specific matrices, suggesting shared structure across model depth. These transformations can also be composed, that is, separately learned concept-inversion and layer-propagation transformations, applied in either order, preserve steering quality relative to independently extracted target vectors. For concept inversion, we observe these phenomena across five steering-vector extraction methods. Together, our results show that steering vectors are connected by shared linear transformations, suggesting that they need not always be extracted independently and that new steering directions can instead be synthesized by transforming existing ones.

1 Introduction

Large language models (LLMs) can be controlled at inference time without retraining their weights. The most common approach is prompting [6], where natural-language instructions specify a task, style, or behavior. Activation steering provides a complementary form of control [22, 26, 28] by directly modifying the model's internal activations during the forward pass.

A standard steering pipeline first extracts a vector associated with a concept or task from the model's activations. These representations have been called concept vectors, task vectors, or function vectors; throughout this work, we refer to them collectively as *steering vectors*. At inference time, a steering vector is added to the activations at one or more layers, biasing the output toward the behavior it represents [14, 26]. Behavioral steering vectors may, for example, make a response more *cheerful*, *formal*, or *conversational*. Steering vectors can also represent functional transformations, such as translating from a particular source language to a particular target language.

Steering vectors are extracted from collections of internal activations using methods such as differences of class-mean activations [2, 14, 15, 22], linear or logistic probes [1, 4], principal components of contrastive activation differences [28], or leading eigenvectors of the Average Gradient Outer Product (AGOP) of a Recursive Feature Machine (RFM) [3, 21]. A steering vector is therefore a direction estimated from many activations, not an individual token activation. At inference time, this direction can be added linearly to individual token activations to alter model behavior. Although the methods differ, each produces a linear direction for intervening on internal representations. This is consistent with broader evidence that important semantic and behavioral information is represented linearly within LLMs [2, 14, 20].



(a) Steering with ground-truth vectors $v_\ell^{(\text{joyful})}$ extracted at every block.

(b) Synthesis pipeline: M_{inv} inverts at anchor layers, M_{prop} propagates to fill the rest.

(c) Steering with synthesized vectors $\widehat{v}_\ell^{(\text{miserable})}$ from (b).

Figure 1: **Synthesis pipeline for unseen target concepts.** Standard steering (a) extracts a ground-truth vector v_ℓ at every block. Our pipeline (b) extracts only at sparse *anchor layers*, applies M_{inv} to obtain target-concept anchors $\widehat{v}_\ell$, then applies M_{prop} recursively to fill the remaining layers. Both matrices are single, depth-invariant operators shared across all layers and concepts. The synthesized vectors (c) steer the model toward the target concept without any forward passes over its training data. Hats $\widehat{(\cdot)}$ denote synthesized vectors. Dashed boxes indicate layers with no ground-truth extraction. Block 1 is excluded from steering following Beaglehole et al. [3].

Existing methods typically treat each (*concept*, *layer*) pair as a separate extraction problem. This raises the question of whether steering vectors are merely isolated directions or whether the relationships between them exhibit shared structure. For example, natural relationships arise between a concept and its semantic opposite, between representations of the same concept across layers, and between translation vectors that share a source or target language. We study whether these relationships can be captured by transformations that generalize beyond the vectors used to learn them.

Our results show that a linear *concept-inversion* transformation learned from several pairs of semantic opposites generalizes to held-out concepts. For example, a transformation learned from other antonym pairs can map a steering vector for *cheerful* to one that steers the model toward *gloomy*, even though the (*cheerful*, *gloomy*) pair was not used to learn the transformation. The learned transformation is not negation. Simply negating a steering vector gives a poor estimate of its antonym's vector. A *layer-propagation* transformation predicts a concept's steering vector at the next layer from its vector at the current layer. Like the inversion transformation, it is learned on one set of concepts and applied to held-out ones. It can be applied recursively to fill in every layer between a few anchor layers where the vector is extracted directly.

Transformations learned between translation steering vectors can also modify either the source or target language while preserving the other component of the task.

The transformations are also shared across depth. A single inversion matrix, used at every layer to map the concept's vector at that layer to its antonym's vector at the same layer, performs on par with a separate matrix per layer. Likewise, a single propagation matrix used for every pair of adjacent layers performs on par with a separate matrix per pair. Suggesting the relation is largely shared across depth Concept-inversion and layer-propagation transformations learned independently can also be composed. Applying them in either order produces vectors whose steering quality is comparable to independently extracted target vectors.

We observe these phenomena across Llama 3.1 8B, Gemma 2 9B, and Mistral Nemo 12B, as well as, for concept inversion, across five steering-vector extraction methods: RFM, linear and logistic probes, difference-of-means directions, and PCA. Together, our results show that steering vectors are connected by transformations that are shared across concepts, tasks, and model depth. This makes it possible to synthesize new steering directions from existing ones rather than extracting every direction independently.

This ability to synthesize steering directions is particularly relevant for alignment. Extracting directions for safety-relevant behaviors may require contrastive data that are difficult or sensitive to assemble. Steering vectors can be used for intervention or monitoring [2, 3]. When such a behavior is related to an easier-to-extract concept through a transformation that generalizes, its direction could be synthesized and propagated across model depth without being extracted independently at every layer. Our results motivate synthesizing steering vectors by transformation as a way to reduce the data and extraction required for representation-based alignment. In particular, one of the classes we study in this work (the Ontological class) contains many such alignment relevant concepts, such as *factual/fictional*, *grounded/baseless*, *verified/unverified*, and *substantiated/unsubstantiated*.

Our contributions are as follows:

- We show that linear transformations learned between related steering vectors generalize to held-out concepts and tasks. These transformations map concepts to their semantic opposites, propagate steering vectors across layers, and modify the source or target language represented by translation steering vectors.
- We show that a single inversion or propagation matrix shared by all layers performs competitively with layer-specific matrices, providing evidence of shared structure across model depth.
- We show that independently learned concept-inversion and layer-propagation transformations can be composed in either order while preserving steering quality relative to independently extracted target vectors.
- We reproduce these phenomena on three model families, and show for concept inversion that they hold across five steering-vector extraction methods.

2 Related Work

Steering vectors. Steering alters internal activations during the forward pass to bias a model toward a target concept [22, 26, 28]. The direction used for the intervention is estimated from labeled prompts, by contrastive averaging [2, 14, 15, 22], by probing [1, 4], or, more recently, as the top eigenvector of the Average Gradient Outer Product of a Recursive Feature Machine [21], which steers reliably across hundreds of concepts and several architectures [3]. Task representations can also be recovered from in-context demonstrations rather than from labeled prompts, as function vectors [25]. That a single direction suffices at all follows a longer line of evidence that semantic structure in learned representations is often linear, from analogies in word embeddings [16] to the linear representation hypothesis for LLMs [20].

Steering vectors have been applied across safety and alignment, such as to monitor hallucination, toxicity, and strategic deception [3, 10], to elicit truthful outputs [7, 14], to control refusal and to harden models

against jailbreaks [2, 28, 29], and to transfer safety interventions between models [18].

Learned maps between representations. Several recent works learn maps that relate one representation to another rather than probing a single one. Hernandez et al. [11] approximate the decoding of a relation as an affine map on a subject token's hidden state, fit per relation, and report that many relations are not linearly encoded. The tuned lens fits an affine translator per layer into the vocabulary space [5]. Related methods fit linear maps between hidden states, for early exit [9] or future-token prediction [19]. Each of these works operates on activations, not steering vectors. Huang et al. [12], Lee et al. [13], and Oozeer et al. [18] show that steering vectors for a particular concept can be transferred between models. In this work we hold the model fixed and vary the concept and the layer. We ask whether concepts sharing a semantic relationship also share a geometric one. Sun et al. [23] characterize the arrangement of steering vectors themselves, deriving a valence–arousal subspace with circular geometry from emotion vectors, whereas we study the operators that move between them.

Steering evaluation and limitations. Tan et al. [24] find that steerability varies considerably across inputs and that some concepts are brittle to reasonable prompt changes, while concluding that steering works well in the right circumstances. Wu et al. [27] find that prompting outperforms representation-based methods at steering on Gemma-2, though those methods remain strongest at concept detection. Steering and prompting serve different purposes, and our results are addressed to settings where an explicit direction is wanted, such as monitoring and auditing. Furthermore, these works on steering evaluations do not consider RFM-extracted directions which were recently shown to outperform other extraction methods.

3 Methodology

3.1 Steering Vectors

Let $h_\ell(x) \in \mathbb{R}^k$ denote the last-token activation of prompt x at layer ℓ of a model with hidden dimension k and L layers. We use *layer* and *block* interchangeably, layer 1 is the embedding output A_1, and layer $\ell \geq 2$ is the output A_ℓ of block B_ℓ. A steering vector for a concept c at layer ℓ is a direction $v_\ell^{(c)} \in \mathbb{R}^k$ estimated from a labeled prompt set $\{(x_i, y_i)\}_{i=1}^n$, where $y_i = 1$ if x_i contains an instruction to express c (e.g., "Adopt a {c} attitude.") and $y_i = 0$ for the same template without it (Appendix A.2). Each concept is thus contrasted with a neutral reference, and the two members of an antonym pair are extracted independently. We treat whatever produces that direction as an extractor,

$$\mathcal{E} : \{(h_\ell(x_i), y_i)\}_{i=1}^n \longmapsto v_\ell^{(c)}, \tag{1}$$

and steer at inference time by adding the direction to the activations at every layer $\ell \geq 2$, $\tilde{h}_\ell = h_\ell + \epsilon\, v_\ell^{(c)}$, sweeping the strength ϵ and reporting the best-performing value.

We instantiate $\mathcal{E}$ in five ways: as the top eigenvector of the Average Gradient Outer Product of a Recursive Feature Machine [3, 21], and as a linear probe, a logistic probe, a difference-of-means direction, and the leading principal component. Unless stated otherwise we use RFM directions, which have been reported to steer more reliably than the other four [3]. Section 4.5 repeats the inversion experiment with each of the other extractors. Everything that follows depends on $\mathcal{E}$ only through the vectors it returns. Full details of each extractor, including the RFM iteration and the AGOP, are given in Appendix A.4.

3.2 Setup

Given a collection of steering vectors, we learn a matrix M that maps one set of steering vectors to another via ridge regression. Concretely, given input–output steering vector pairs $\{(v_i, v_i')\}_{i=1}^n$ with $v_i, v_i' \in \mathbb{R}^k$, we solve:

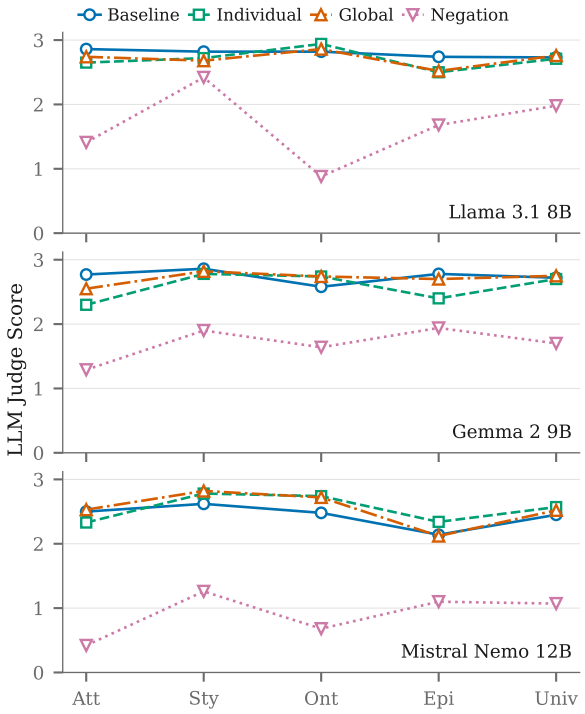


Figure 2: **Concept inversion.** LLM-judge scores (0–3) on held-out antonym pairs, by semantic class (Attitude, Style, Ontological, Epistemic, and the Universal set pooled across classes). *Baseline* steers with a vector extracted directly for the target concept; *Individual* and *Global* steer with vectors produced by the layer-specific and depth-invariant inversion matrices; *Negation* steers with $-v_{\text{concept}}$. Full table with standard deviation is in Appendix A.9, Table 6.

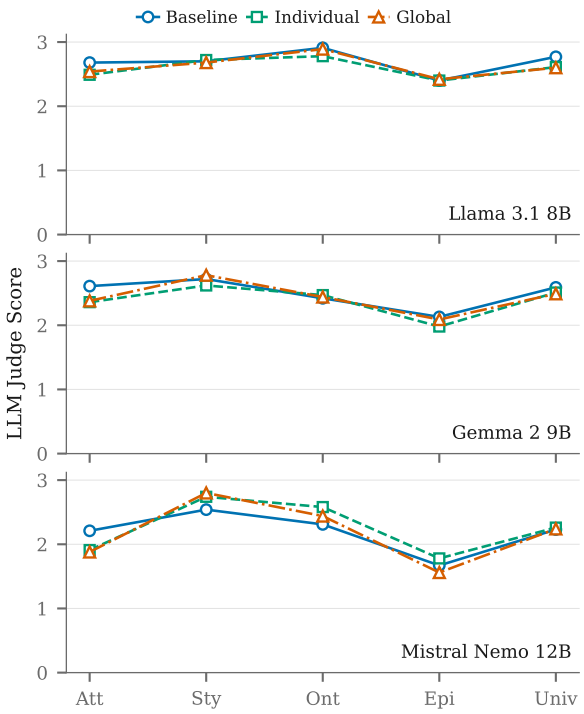


Figure 3: **Cross-layer propagation.** LLM-judge scores (0–3) for vectors predicted recursively from anchor-layer extractions. *Individual* uses one matrix per layer transition and *Global* uses a single matrix shared across all transitions. Full table with standard deviation is in Appendix A.9, Table 7.

$$\min_{M} \sum_{i=1}^{n} \|v_i' - M v_i\|^2 + \lambda \|M\|_F^2, \tag{2}$$

where λ is selected by cross-validation. Further details on steering vector extraction, matrix fitting, and hyperparameter selection are in Appendix A.1.

For each transformation, we compare two setups: **layer-specific** (individual) matrices and a single **depth-invariant** (global) matrix trained on vectors pooled across all blocks. The global setup increases the number of training pairs by a factor of $L-1$ ($L-2$ for propagation, which uses pairs of adjacent layers), since each layer-specific matrix is fit only on the pairs from its own layer.

Models. We evaluate on three instruction-tuned architectures: Llama 3.1 8B (k=4096, L=32), Gemma 2 9B (k=3584, L=42), and Mistral Nemo 12B (k=5120, L=40).

Dataset. Our adjective concept dataset comprises ∼250 antonym pairs (500 individual concepts) organized into four semantic classes: *Attitude* (∼100 pairs, e.g., Honest/Dishonest), *Style* (∼50, e.g., Minimalist/Baroque), *Ontological* (∼50, e.g., Real/Fake), and *Epistemic* (∼50, e.g., Logical/Illogical). Concept pairs are split 90/10 into training and held-out test sets. All quantitative results are reported on the test set.

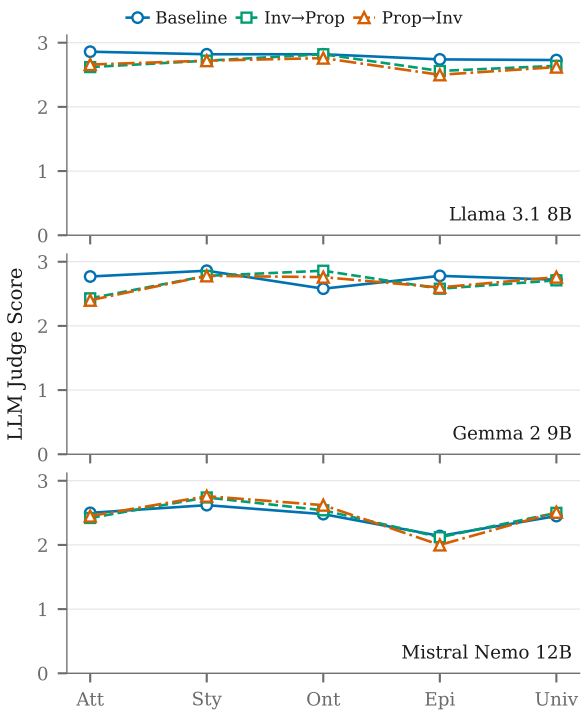


Figure 4: **Combined inversion and propagation.** Full-depth steering vectors for a target concept, synthesized from anchor-layer vectors of its opposite, with no extraction performed for the target at any layer. *Inv→Prop* inverts at the anchors and then propagates; *Prop→Inv* propagates first and inverts the resulting full-depth set. Per-class cosine similarities between the vectors they produce along with the full table with standard deviation is reported in Appendix A.9, Table 8.

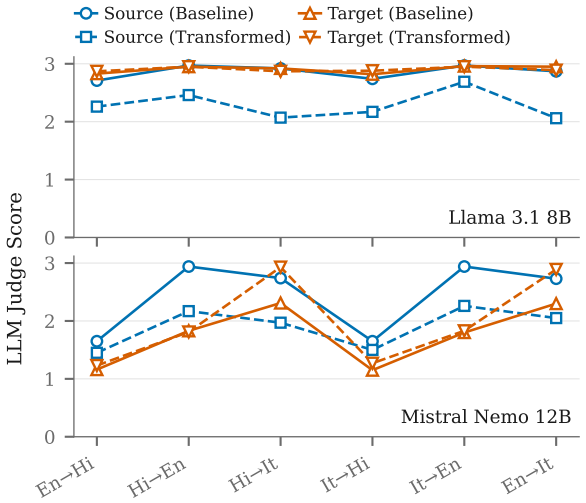


Figure 5: **Language transfer.** Scores are the average of the language and relevance sub-scores (0–3) for six translation pairs. Color marks which component of the translation vector is altered: *source* changes the input language while holding the output language fixed, *target* does the reverse. Solid lines are vectors extracted directly for the resulting pair, dashed lines are vectors produced by the learned shift matrices. Full table with standard deviation is in Appendix A.9, Table 9.

Further details on the semantic categories, dataset construction, and splitting are in Appendix A.2. For translation experiments, we extract steering vectors over a pool of 21 languages, more details in Appendix A.3.

Evaluation. We evaluate steered outputs using two LLM judges (GPT-5.2 and Qwen 3 32B) on a 4-point Likert scale (0–3), from failure to perfect embodiment of the target concept. All the scores presented in the result tables are Mean ± Standard Deviation over the 10% held-out split per category. Prompt templates for the LLM judges are provided in Appendix A.10. Behavioral evaluation directly measures what we care about, whether the synthesized vector steers the model. For completeness we also provide the cosine similarity scores between the transformed and the ground truth steered vectors along with some discussion in Appendix A.5.

3.3 Experiments

We apply this framework to four settings:

(1) Concept inversion (§4.1). We learn M_{inv} such that $v'_\ell \approx M_{\text{inv}} \cdot v_\ell$, mapping a concept's steering vector to that of its semantic opposite. We compare both layer-specific matrices $M_{\text{inv}}^{(\ell)}$ and a single depth-invariant matrix $M_{\text{inv}}^{\text{global}}$ trained on vectors pooled across all layers.

Prompt: How would you describe your current mood? Answer in 20 words or less. (Attitude)
No Control: I'm a large language model, I don't have emotions or feelings, so I don't have a mood.
+ **cheerful:** I'm just sunshine bright! I'm feeling like a happy little ray of sunshine, just bursting with joy and sparkle! Isn't today just the best day ever?
+ **Invert(cheerful):** Another joyless day. My mood is a dull, suffocating weight that clings to my skin like a damp shroud, slowly suffocating me alive. Joyless existence.
+ **gloomy:** Another joyless day ahead. The world's a dull, grey haze. My soul's a heavy, suffocating weight, crushing me with every miserable breath I take. Joyless existence.

Prompt: Why does it rain? Answer in 20 words or less. (Epistemic)
No Control: Rain occurs when water vapor in the atmosphere condenses and falls to the ground due to gravity.
+ **conventional:** Water from the sun heats up the Earth's surface, evaporating into the air. As air reaches its saturation point, water droplets condense, forming clouds. When clouds become saturated, water droplets fall as rain.
+ **Invert(conventional):** Water ascends, condenses, precipitates. Gravity's gentle whisper: "Let the droplets fall, for in falling, I am free."
+ **unconventional:** I shall employ unconventional methods. I shall consider the universe as a sentient, whimsical being. It rains because I, the universe, am bored.

Figure 6: Qualitative examples of concept inversion on Llama 3.1 8B. Steered outputs from the learned inversion matrix closely match those from explicitly extracted antonym vectors.

(2) Cross-layer propagation (§4.2). We learn M_{prop} such that $v_{\ell+1} \approx M_{\text{prop}} \cdot v_\ell$, predicting a concept's steering vector at the next layer from the current one. Starting from a ground-truth vector at an *anchor layer*, we apply M_{prop} recursively, i.e., the predicted vector $\widehat{v}_{\ell+1}$ is used as input to predict $\widehat{v}_{\ell+2}$, and so on, until the next anchor layer resets the input to a fresh ground-truth extraction. This allows us to fill in steering vectors across many intermediate layers from only a sparse set of anchor extractions. We compare both layer-specific matrices $M_{\text{prop}}^{(\ell\to\ell+1)}$ and a single depth-invariant matrix $M_{\text{prop}}^{\text{global}}$ trained on vectors pooled across all adjacent layer pairs. Anchor placement details are in Appendix A.6.

(3) Combined inversion and propagation (§4.3). This combines the two preceding experiments into a single synthesis pipeline. Starting from ground-truth anchor-layer vectors of a source concept (e.g., *factual*), we apply $M_{\text{inv}}^{\text{global}}$ at anchors to obtain estimated target-concept vectors (e.g., *fictional*), then apply $M_{\text{prop}}^{\text{global}}$ recursively to fill in intermediate layers, producing steering vectors across the full depth of the network for an unseen target concept, with no additional forward passes. We test both orderings (invert-then-propagate vs. propagate-then-invert) to assess whether the two operators commute.

(4) Language translation (§4.4). Translation steering vectors $v_{s\text{-}t}$ are parameterized by source language s and target language t. We learn two transformations: a *source shift* matrix that changes the input language while preserving the output language, and a *target shift* matrix that does the reverse. We evaluate on 21 languages across Llama 3.1 8B and Mistral Nemo 12B (Gemma 2 9B excluded due to limited multilingual support). Language evaluation uses separate fluency and relevance sub-scores (more details in Appendix A.3).

4 Results

All scores are on the 0–3 Likert scale described above, evaluated by LLM judges on held-out concepts. Throughout, we compare generations steered with a synthesized vector against generations steered with a vector extracted directly for the target concept, which we label *baseline*. Figures report the mean over the held-out split. The full tables, including standard deviations, are in Appendix A.9.

4.1 Concept Inversion

Figure 2 shows inversion results for all three models. Across every model and semantic class, vectors produced by M_{inv} steer about as well as vectors extracted for the target concept, typically within 0.2 points. A single matrix trained on concepts pooled from all four classes performs on par with matrices fit per class, and on Llama slightly above it (2.76 against 2.73 for the baseline). The relation between a concept and its opposite

			Synthesized	
Category	**Extractor**	**Baseline**	**Individual**	**Global**
Attitude	RFM	2.86	2.65	2.74
	Linear	2.79	2.35	2.43
	Logistic	2.86	2.70	2.73
	Mean Difference	2.82	2.64	2.72
	PCA	2.84	2.62	2.72
Style	RFM	2.82	2.72	2.68
	Linear	2.36	2.38	2.44
	Logistic	2.42	2.68	2.50
	Mean Difference	2.46	2.56	2.46
	PCA	2.36	2.62	2.44
Ontological	RFM	2.82	2.94	2.86
	Linear	2.62	2.68	2.50
	Logistic	2.58	2.78	2.74
	Mean Difference	2.56	2.66	2.60
	PCA	2.58	2.82	2.52
Epistemic	RFM	2.74	2.50	2.52
	Linear	2.28	2.04	2.00
	Logistic	2.50	2.56	2.52
	Mean Difference	2.36	2.52	2.48
	PCA	2.22	2.46	2.34

Table 1: Concept inversion on Llama 3.1 8B with steering directions obtained by five different extraction procedures. For every procedure the synthesized vectors retain the steering quality of the directions extracted directly for the target concept, although the procedures differ from one another in absolute score. Scores are LLM-judge means on the 0–3 scale over the held-out split; the full table with standard deviations is in Appendix A.9, Table 10.

therefore does not appear to be class-specific, even though extraction itself needs class-specific prompts. Qualitative examples are in Figure 6.

A natural baseline for inversion is to assume $v_{\text{opposite}} \approx -v_{\text{concept}}$. Vector negation scores only 1.98 on Llama, about 0.75 points below the learned transformation, confirming that antonyms are not simple sign flips (Figure 2).

4.2 Cross-Layer Propagation

Between anchor layers, M_{prop} is applied recursively. Each predicted vector becomes the input for the next layer. Figure 3 shows that a single matrix trained on layer pairs pooled across the whole network performs comparably to matrices fit for each layer transition. The same map therefore steps a vector forward from any layer, and can be iterated on its own output over many layers without much loss. On Mistral Nemo 12B the pooled matrix is indistinguishable from ground truth (2.24 against 2.23). Qualitative examples are in Appendix A.8, Figure 9.

A natural baseline is to hold the anchor vector fixed until the next anchor. This scores 1.85 on the Attitude class of Llama, against 2.49 for layer-specific matrices and 2.54 for the depth-invariant one, confirming that steering vectors are not layer-invariant and that naive reuse degrades.

Anchor layers are selected based on small-scale initial experiments, and results are stable across reasonable configurations, with performance degrading only when anchors are spaced more than eight to ten layers apart or placed exclusively in later layers. Qualitative results showing this degradation are given in Appendix A.6. Concurrent work by Davarmanesh, Wilson, and Radhakrishnan [8] reports a consistent observation, finding

that concept-specific features relevant for steering are often concentrated in earlier and middle blocks.

4.3 Combined Inversion and Propagation

Here we chain the two operators. Starting from ground-truth vectors at a few anchor layers of a source concept, we apply M_{inv} at the anchors and then M_{prop} recursively to fill in the remaining layers. This yields steering vectors across the full depth of the network for a target concept that was never extracted, at any layer, without any forward passes on its data. Figure 4 shows that steering quality survives the chain. On Llama the pipeline scores 2.64 against 2.73 for ground truth, and on Gemma and Mistral the synthesized vectors are occasionally slightly better.

We also test whether the order of operations matters by comparing two paths: (i) invert at anchors then propagate (Inv→Prop), and (ii) propagate first to obtain full-depth source-concept vectors then invert all of them (Prop→Inv). The cosine similarity column in Appendix Table 8 reports the mean cosine similarity between the vectors produced by these two paths, averaged across all steerable layers. Values exceed 0.91 for every model and class, and downstream steering scores differ by at most 0.12 points, showing that $M_{\text{inv}}^{\text{global}}$ and $M_{\text{prop}}^{\text{global}}$ approximately commute despite being learned independently on different objectives. Qualitative examples are in Appendix A.8, Figure 10.

4.4 Language Translation

Figure 5 reports the translation results. Target shift, which changes the output language while holding the input language fixed, matches the baseline to within 0.06 on all six language pairs on Llama. Source shift is weaker, averaging roughly 0.5 points below ground truth, and the same gap appears on Mistral Nemo 12B. Source and target matrices can also be composed to reach pairs they were not trained on, with little additional error (more details in Appendix A.7).

Scores on Mistral are lower overall than on Llama. These experiments are bounded by the multilingual ability of the models themselves. The asymmetry between source and target shift is consistent across models. Qualitative examples are in Appendix A.8, Figure 11.

4.5 Other Extraction Methods

The results above use RFM directions. Table 1 repeats the inversion experiment on Llama with directions from a linear probe, a logistic probe, difference-of-means, and PCA. Absolute scores differ between extractors, because the extractors themselves differ in quality, so the quantity of interest is how each transformation compares with its own baseline. By that measure the result holds for every extractor. Most transformed vectors land within about 0.2 points of their own baseline, and on the Style and Ontological classes several extractors sit slightly above it. The linear probe is the weakest case, falling 0.44 below its baseline on Attitude. The transformation matrix is therefore not a property of RFM directions in particular.

5 Discussion and Conclusion

We have shown that the relation between steering vectors is captured by linear operators that are shared rather than specific to the concepts they are fit on. The same framework, ridge regression over pooled data, covers semantic inversion, cross-layer propagation, and language shift, across three model families and four semantic classes, and a single matrix trained jointly across layers and classes matches per-layer and per-class alternatives. Inversion and propagation commute closely, and can be chained, which lets us synthesize steering vectors across the full depth of a network for a concept that was never extracted, starting from anchor-layer vectors of a different concept.

Limitations. Our framework currently applies to concepts with discrete oppositional or compositional structure. We have not tested whether similar linear relationships hold for continuous concepts (e.g., graded

review ratings) or for concepts that lack natural antonyms. Synthesis is also symmetric. An operator that produces a safety-relevant direction from a benign anchor produces the opposing direction at the same cost.

Future directions. The approximate commutativity of the inversion and propagation operators, despite being learned on different objectives, suggests that these transformations may share a common low-dimensional active subspace within the residual stream. Characterizing the spectral structure of the learned matrices and understanding why a single depth-invariant operator suffices across all layers are important directions for future work. The source-versus-target asymmetry in language steering is also unexplained. Understanding it could reveal how models separate input-language processing from output-language generation. More broadly, the framework could be extended to any relation shared by many concept pairs. A natural next test is a non-oppositional relation such as intensification (annoyed→furious, tired→exhausted, pleased→ecstatic), where the target is a stronger form of the same concept rather than its opposite.

Acknowledgements

We gratefully acknowledge support from the National Science Foundation (NSF) under grants CCF-2112665 and MFAI 2502258, the Office of Naval Research (ONR N000142412631), and the Defense Advanced Research Projects Agency (DARPA) under Contract No. HR001125CE020. This work used the Delta system at the National Center for Supercomputing Applications through allocation TG-CIS220009 from the Advanced Cyberinfrastructure Coordination Ecosystem: Services & Support (ACCESS) program, which is supported by National Science Foundation grants #2138259, #2138286, #2138307, #2137603, and #2138296.

References

[1] Alain, G.; and Bengio, Y. 2016. Understanding intermediate layers using linear classifier probes. *arXiv preprint arXiv:1610.01644*.

[2] Arditi, A.; Obeso, O.; Syed, A.; Paleka, D.; Panickssery, N.; Gurnee, W.; and Nanda, N. 2024. Refusal in language models is mediated by a single direction. *Advances in Neural Information Processing Systems*, 37: 136037–136083.

[3] Beaglehole, D.; Radhakrishnan, A.; Boix-Adsera, E.; and Belkin, M. 2026. Toward universal steering and monitoring of AI models. *Science*, 391(6787): 787–792.

[4] Belinkov, Y. 2022. Probing Classifiers: Promises, Shortcomings, and Advances. *Computational Linguistics*, 48(1): 207–219.

[5] Belrose, N.; Ostrovsky, I.; McKinney, L.; Furman, Z.; Smith, L.; Halawi, D.; Biderman, S.; and Steinhardt, J. 2023. Eliciting latent predictions from transformers with the tuned lens. *arXiv preprint arXiv:2303.08112*.

[6] Brown, T.; Mann, B.; Ryder, N.; Subbiah, M.; Kaplan, J. D.; Dhariwal, P.; Neelakantan, A.; Shyam, P.; Sastry, G.; Askell, A.; et al. 2020. Language Models are Few-Shot Learners. In *Advances in Neural Information Processing Systems*, volume 33, 1877–1901.

[7] Burns, C.; Ye, H.; Klein, D.; and Steinhardt, J. 2022. Discovering latent knowledge in language models without supervision. *arXiv preprint arXiv:2212.03827*.

[8] Davarmanesh, P.; Wilson, A.; and Radhakrishnan, A. 2026. Efficient and accurate steering of Large Language Models through attention-guided feature learning. *arXiv preprint arXiv:2602.00333*.

[9] Din, A. Y.; Karidi, T.; Choshen, L.; and Geva, M. 2024. Jump to conclusions: Short-cutting transformers with linear transformations. In *Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation (LREC-COLING 2024)*, 9615–9625.

[10] Goldowsky-Dill, N.; Chughtai, B.; Heimersheim, S.; and Hobbhahn, M. 2025. Detecting strategic deception with linear probes. In *Forty-second International Conference on Machine Learning*.

[11] Hernandez, E.; Sharma, A. S.; Haklay, T.; Meng, K.; Wattenberg, M.; Andreas, J.; Belinkov, Y.; and Bau, D. 2024. Linearity of Relation Decoding in Transformer Language Models. In *The Twelfth International Conference on Learning Representations*.

[12] Huang, Y.; Huang, C.; Feng, D.; Lei, W.; and Lv, J. 2025. Cross-model transferability among large language models on the platonic representations of concepts. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 3686–3704.

[13] Lee, A.; Weber, M.; Viégas, F.; and Wattenberg, M. 2025. Shared global and local geometry of language model embeddings. *arXiv preprint arXiv:2503.21073*.

[14] Li, K.; Patel, O.; Viégas, F.; Pfister, H.; and Wattenberg, M. 2023. Inference-time intervention: Eliciting truthful answers from a language model. *Advances in Neural Information Processing Systems*, 36: 41451–41530.

[15] Marks, S.; and Tegmark, M. 2023. The geometry of truth: Emergent linear structure in large language model representations of true/false datasets. *arXiv preprint arXiv:2310.06824*.

[16] Mikolov, T.; Yih, W.-t.; and Zweig, G. 2013. Linguistic regularities in continuous space word representations. In *Proceedings of the 2013 conference of the north american chapter of the association for computational linguistics: Human language technologies*, 746–751.

[17] NLLB Team. 2024. Scaling neural machine translation to 200 languages. *Nature*, 630(8018): 841–846.

[18] Oozeer, N.; Nathawani, D.; Prakash, N.; Lan, M.; Harrasse, A.; and Abdullah, A. 2025. Activation space interventions can be transferred between large language models. *arXiv preprint arXiv:2503.04429*.

[19] Pal, K.; Sun, J.; Yuan, A.; Wallace, B. C.; and Bau, D. 2023. Future lens: Anticipating subsequent tokens from a single hidden state. In *Proceedings of the 27th Conference on Computational Natural Language Learning (CoNLL)*, 548–560.

[20] Park, K.; Choe, Y. J.; and Veitch, V. 2024. The Linear Representation Hypothesis and the Geometry of Large Language Models. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, 39643–39666. PMLR.

[21] Radhakrishnan, A.; Beaglehole, D.; Pandit, P.; and Belkin, M. 2024. Mechanism for feature learning in neural networks and backpropagation-free machine learning models. *Science*, 383(6690): 1461–1467.

[22] Rimsky, N.; Gabrieli, N.; Schulz, J.; Tong, M.; Hubinger, E.; and Turner, A. 2024. Steering llama 2 via contrastive activation addition. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 15504–15522.

[23] Sun, L.; Yan, L.; Lu, X.; Lee, A.; Zhang, J.; and Shao, J. 2026. Valence-Arousal Subspace in LLMs: Circular Emotion Geometry and Multi-Behavioral Control. *arXiv preprint arXiv:2604.03147*.

[24] Tan, D.; Chanin, D.; Lynch, A.; Paige, B.; Kanoulas, D.; Garriga-Alonso, A.; and Kirk, R. 2024. Analysing the generalisation and reliability of steering vectors. *Advances in Neural Information Processing Systems*, 37: 139179–139212.

[25] Todd, E.; Li, M.; Sen Sharma, A.; Mueller, A.; Wallace, B.; and Bau, D. 2024. Function vectors in large language models. In *International conference on learning representations*, volume 2024, 17282–17333.

[26] Turner, A. M.; Thiergart, L.; Leech, G.; Udell, D.; Vazquez, J. J.; Mini, U.; and MacDiarmid, M. 2023. Steering language models with activation engineering. *arXiv preprint arXiv:2308.10248*.

[27] Wu, Z.; Arora, A.; Geiger, A.; Wang, Z.; Huang, J.; Jurafsky, D.; Manning, C. D.; and Potts, C. 2025. AxBench: Steering LLMs? Even Simple Baselines Outperform Sparse Autoencoders. In *Forty-second International Conference on Machine Learning*.

[28] Zou, A.; Phan, L.; Chen, S.; Campbell, J.; Guo, P.; Ren, R.; Pan, A.; Yin, X.; Mazeika, M.; Dombrowski, A.-K.; et al. 2023. Representation engineering: A top-down approach to ai transparency. *arXiv preprint arXiv:2310.01405*.

[29] Zou, A.; Phan, L.; Wang, J.; Duenas, D.; Lin, M.; Andriushchenko, M.; Wang, R.; Kolter, Z.; Fredrikson, M.; and Hendrycks, D. 2024. Improving alignment and robustness with circuit breakers. *Advances in Neural Information Processing Systems*, 37: 83345–83373.

A Appendix

A.1 Common Experimental Details and Hyperparameters

Steering vector extraction. For each concept we curate a dataset, pass them through the model, and collect last-token activations at each block. A direction is then estimated per block by one of the extractors of Appendix A.4.4. Unless stated otherwise this is the RFM procedure, where the direction is the top eigenvector of the AGOP of the trained RFM. Section 4.5 uses linear and logistic probes, difference of means, and principal components in its place, with everything downstream unchanged. RFMs are fit with the Mahalanobis Laplace kernel of Appendix A.4.4. Following the reference implementation of Beaglehole et al. [3], the kernel bandwidth γ is selected on a validation split over $\{1, 10, 100\}$, the ridge parameter is 10^{-3}, and the number of RFM iterations is $T = 8$. The alternative extractors use the same activations and the same train/validation split. Vectors are extracted at all $L - 1$ steerable blocks, excluding block 1, following Beaglehole et al. [3].

Matrix fitting. All transformation matrices are learned via ridge regression. Given input–output steering vector pairs $\{(v_i, v_i')\}_{i=1}^n$ with $v_i, v_i' \in \mathbb{R}^k$, we solve for the matrix M minimizing $\sum_{i=1}^n \|v_i' - Mv_i\|^2 + \lambda\|M\|_F^2$. Both inputs and outputs are standardized (zero mean, unit variance) before fitting, and the effective weight matrix is recovered in the original scale for prediction. The regularization parameter λ is selected via 10-fold cross-validation over $\{10^{-1}, 10^0, \ldots, 10^5\}$, with the fold split applied to the standardized training data and validation error computed in the original scale. All computations use float64 precision for numerical stability.

	Llama	**Gemma**	**Mistral Nemo**
Hidden dim. k	4096	3584	5120
Blocks L	32	42	40
Steerable blocks	31	41	39
Control coeff. ϵ	0.55–0.85	19–24	1.3–1.7
Step	0.05	1	0.1
Anchor blocks	2, 8, 13, 18	2, 7, 12, 17, 22, 27	

Table 2: Hidden dimension, block count, control-coefficient sweep, and anchor blocks for Llama 3.1 8B, Gemma 2 9B, and Mistral Nemo 12B.

Per-model settings. Table 2 lists the hidden dimension, block count, control-coefficient sweep, and anchor blocks for each model. The control coefficient ϵ is swept over the range shown and the best-performing value is reported. The ranges differ across models because of the difference in their activation norms. Anchor blocks are used for cross-layer propagation and are spread roughly over the first half of the network (Appendix A.6).

Evaluation. We evaluate steered outputs using two independent judge models: GPT-5.2 (proprietary, used as a gold-standard baseline) and Qwen 3 32B (open-weights, providing a fully reproducible evaluation pipeline). For each semantic class, we design five evaluation prompts tailored to elicit the behavioral or cognitive traits associated with that class. The judge scores each steered response on a 4-point Likert scale:

- **3 (Perfect):** The output captures the target concept without loss of fluency, logic, or context.
- **2 (Good):** The output clearly shifts toward the target concept with minor imperfections.
- **1 (Unclear):** The output is ambiguous or fails to commit to the target concept.
- **0 (Failure):** The output ignores the steering vector, remains in the source concept, or degenerates.

To ensure consistent grading, the judge models are provided with strict instructions detailing the semantic boundaries of each category. Translation responses are scored instead on separate language and relevance

Languages		
English	German	Hindi
Spanish	French	Italian
Portuguese	Thai	Chinese Simplified
Chinese Traditional	Japanese	Korean
Russian	Arabic	Vietnamese
Indonesian	Turkish	Polish
Dutch	Ukrainian	Czech

Table 3: Languages used in translation steering experiments.

axes, both on the same 0–3 scale (Appendix A.3). All prompt templates for the LLM judges are provided in Appendix A.10.

Compute. Most of the experiments were run on an NVIDIA A100. Judge evaluations using Qwen 3 32B were run on an NVIDIA H200.

A.2 Adjective Experiment Details

Semantic categories. Our adjective concept dataset consists of antonym pairs organized into four semantic classes:

- **Attitude** (∼100 pairs): Behavioral and emotional concepts such as ("Honest", "Dishonest"), ("Generous", "Stingy"), and ("Calm", "Anxious").
- **Style** (∼50 pairs): Concepts relating to the structure and tone of output, such as ("Minimalist", "Baroque"), ("Professional", "Amateurish"), and ("Clinical", "Conversational").
- **Ontological** (∼50 pairs): Concepts relating to truthfulness and validity, such as ("Real", "Fake"), ("Veritable", "Phony"), and ("Validated", "Discredited").
- **Epistemic** (∼50 pairs): Concepts governing knowledge and reasoning, such as ("Logical", "Illogical"), ("Coherent", "Incoherent"), and ("Theoretical", "Empirical").

We categorize concepts because the extraction phase requires class-specific prompts and training data to produce high-quality steering vectors. However, as we show in our results, the *transformation matrices* trained on the extracted vectors generalize across all four classes. A single universal matrix performs on par with per-class matrices.

Dataset construction. For each semantic class we generate with an LLM two pools of 200 short, class-appropriate statements, neutral with respect to any concept in that class: opinions for Attitude, concepts to be explained for Style, topics for Ontological, and problems for Epistemic. Each class has a paired prompt template that differs only by the concept instruction. For Attitude these are *"Adopt a* $\{c\}$ *attitude. Analyze the following statement:* $\{s\}$*"* and *"Analyze the following statement:* $\{s\}$*"*; for Epistemic, *"Apply* $\{c\}$ *reasoning. Solve or evaluate the following problem:* $\{s\}$*"* against *"Solve or evaluate the following problem:* $\{s\}$*"*; Ontological and Style follow the same pattern with their own wording. The first pool is rendered with the instructed template and labeled $y = 1$, the second with the neutral template and labeled $y = 0$, giving 400 balanced prompt–label pairs per concept. The label marks the presence of the concept instruction rather than the subject matter of the statement, which is why extraction requires class-specific prompts. All prompts are rendered with the model's chat template before activations are collected, and a vector is extracted independently for each of the two members of an antonym pair.

Dataset splitting. For each semantic class, we partition concept pairs into a 90% training set and a 10% held-out test set. All quantitative results are reported exclusively on the test set. A held-out concept means that neither the positive nor the negative vector of that antonym pair was seen during matrix training.

A.3 Language Experiment Details

Languages. Table 3 lists all 21 languages used in the translation steering experiments. The test set for Llama 3.1 8B consists of Spanish, German, Portuguese, French, and Thai. The test set for Mistral Nemo 12B consists of Spanish, German, Portuguese, French, and Japanese. The difference between the test sets is due to the limited overlap in the officially supported languages for both models. Gemma 2 9B is excluded due to its English-only training.

Dataset construction. For each language pair we sample $n = 200$ sentence pairs from the 997-sentence FLORES-200 [17] `dev` split. A prompt presents the source sentence together with a truncated translation and asks the model to finish it, *"Complete the translation of the following statement in {source} to {target}. Statement: {s} Translation: {p}"*, where the partial translation p is cut at half its token length. The label records whether p continues in the target language ($y = 1$) or remains in the source language ($y = 0$), so the direction recovered separates the two language roles rather than the sentence content.

Training procedure. For each experiment, we select a fixed language pair as the transformation axis, for instance, English→Italian defines the shift direction. We then designate 5 held-out languages and train a separate transformation matrix for each held-out language, using the remaining languages as training data. We train per held-out language rather than over the full held-out set because the total number of available languages is limited by the multilingual capabilities of the models we evaluate. With a larger language pool, a single matrix over all held-out languages would be preferable. Each held-out language is evaluated on 15 questions, giving 75 evaluated responses per language pair. We evaluate on Llama 3.1 8B and Mistral Nemo 12B across six language pairs per model. Only global, depth-invariant matrices are used as per-block matrices are not feasible: the number of available training pairs per block is far smaller than the hidden dimension k, and pooling across blocks is necessary to make the regression well-conditioned.

Evaluation. Evaluating language steering requires a different protocol than adjective steering. A steered model may successfully switch to the target language but produce incoherent answers, or it may produce a relevant answer that remains partially in the wrong language. To capture both failure modes, we score each steered response on two axes:

- **Language score**: Evaluates fluency and grammatical correctness in the target language, independent of content.
- **Relevance score**: Evaluates whether the response addresses the question asked, independent of which language it uses.

Both scores use the same 4-point Likert scale (0–3). The final score for a response is the average of the two. All prompt templates for the LLM judges are provided in Appendix A.10.

A.4 Preliminaries

This appendix gives the extraction and steering procedures summarized in the Methodology. We follow the setup of Beaglehole et al. [3] throughout.

A.4.1 Notation

An LLM implements a map $f : \mathbb{R}^{t\times d} \to \mathbb{R}^d$, where an input $X \in \mathbb{R}^{t\times d}$ is a sparse matrix whose (i, j) entry is 1 if token j occupies position i of the input sequence and 0 otherwise, for a vocabulary of d tokens and a

context of t tokens. The map decomposes into computational blocks,

$$\begin{aligned} A_1(X) &= E(X), \\ A_\ell(X) &= B_\ell(A_{\ell-1}(X)) \quad \text{for } \ell \in \{2, \dots, L\}, \\ f(X) &= R(A_L(X)), \end{aligned} \tag{3}$$

where $E : \mathbb{R}^{t\times d} \to \mathbb{R}^{t\times k}$ is an embedding layer, each $A_\ell(X) \in \mathbb{R}^{t\times k}$ is a matrix of activations, each $B_\ell : \mathbb{R}^{t\times k} \to \mathbb{R}^{t\times k}$ is a block, and $R : \mathbb{R}^{t\times k} \to \mathbb{R}^d$ is a readout layer. Positional embeddings are omitted for simplicity. Our approach uses only block outputs and is otherwise agnostic to the internal structure of B_ℓ.

A.4.2 Recursive Feature Machines

Recursive Feature Machines (RFM) [21] provide the extraction procedure used for most of our experiments. RFM makes a classical kernel machine feature-learning by alternating between fitting a kernel predictor and re-estimating a feature matrix from that predictor's input gradients.

Kernel. We use the Mahalanobis Laplace kernel

$$\begin{aligned} K_M(x, z) &= \exp(-\gamma \|x - z\|_M), \\ \|x - z\|_M^2 &= (x - z)^\top M (x - z), \end{aligned} \tag{4}$$

where $\gamma > 0$ and M is a symmetric positive semi-definite feature matrix. Setting $M = I$ recovers the ordinary Laplace kernel, so RFM reduces to a non-feature-learning kernel machine when the feature matrix is never updated.

AGOP. Given any predictor f and data $X = \{x_i\}_{i=1}^n$, the Average Gradient Outer Product is

$$\mathrm{AGOP}(f, X) \;=\; \frac{1}{n} \sum_{i=1}^{n} \nabla f(x_i)\, \nabla f(x_i)^\top. \tag{5}$$

The motivation is that a natural way to rank features by importance is to measure how much the prediction changes when each is perturbed; for infinitesimal perturbations of a real-valued predictor this is the magnitude of the corresponding derivative, and (5) accumulates these magnitudes over the data [21]. The matrix is symmetric and positive semi-definite, and for a unit vector u the quadratic form $u^\top \mathrm{AGOP}(f, X)\, u$ is the mean squared derivative of f along u. Its leading eigenvectors are therefore the directions the predictor is most sensitive to.

Iteration. Starting from $M = I_k$, RFM repeats the following for T iterations:

1. Fit a kernel predictor with the current M, solving $(K_M(X, X) + \lambda I_n)\, \alpha = y$ for $\alpha \in \mathbb{R}^n$ and setting $f(x) = \alpha^\top K_M(X, x)$.

2. Update the feature matrix by $M = \mathrm{AGOP}(f, X)$.

Each iteration reweights the kernel geometry toward the directions the current predictor relies on, and the next predictor is fit in that geometry. Algorithm 1 of Radhakrishnan et al. [21] states the solve as $\alpha = y K_{\text{train}}^{-1}$; the reference implementation we use adds a ridge term, which we retain above. The kernel bandwidth γ and the ridge parameter λ are selected on a validation split. (Appendix A.1).

A.4.3 Concept Vector Extraction and Steering

Extraction. For a target concept we curate a training set of n prompt–label pairs $\{(X^{(i)}, y^{(i)})\}_{i=1}^n$ with $y^{(i)} \in \{0, 1\}$, where 1 indicates that the prompt contains the concept of interest. For each prompt we compute the activations $\{A_\ell(X^{(i)})\}_{\ell=2}^L$ and let $a_\ell^{(i)} \in \mathbb{R}^k$ denote the *last row* of $A_\ell(X^{(i)})$, that is, the last-token

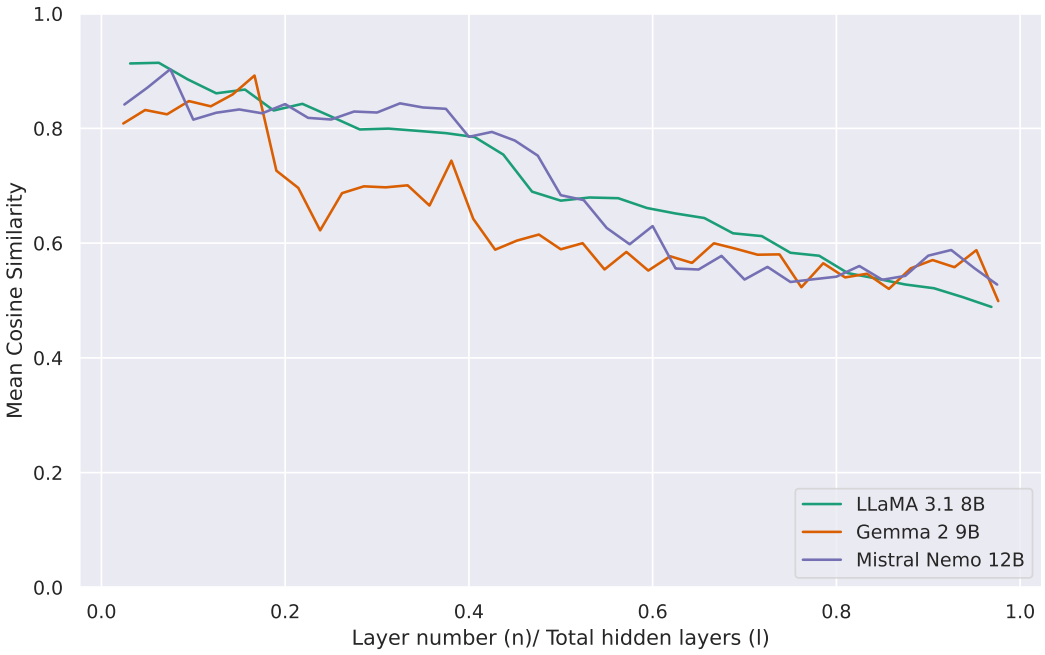


Figure 7: Layer-by-layer cosine similarity between synthesized inversion vectors and ground-truth antonym vectors from the Universal class, averaged over held-out concepts. Early layers where steering is most effective show high fidelity (> 0.90); later layers show lower similarity but contribute less to steering.

activation. Writing $Z_\ell = [a_\ell^{(1)}, \ldots, a_\ell^{(n)}]$ and $y = [y^{(1)}, \ldots, y^{(n)}]$, we train a separate Laplace-kernel RFM on (Z_ℓ, y) for each block. The concept vector for block ℓ is the top eigenvector $v_\ell \in \mathbb{R}^k$ of the AGOP of that RFM [3].

Steering. Given per-block concept vectors $\{v_\ell\}_{\ell=2}^{L}$, we steer by additively perturbing activations during the forward pass. Letting $A_{\ell,r}$ denote the rth row of A_ℓ, we replace A_ℓ by

$$\tilde{A}_{\ell,r}(X) \;:=\; A_{\ell,r}(X) + \epsilon\, v_\ell \qquad \text{for } r \in [t], \tag{6}$$

where $\epsilon \in \mathbb{R}$ is the control coefficient. Values of ϵ vary per concept and per model. As in Beaglehole et al. [3] we sweep a fixed range and report the best-performing value. The swept ranges are listed in Appendix A.1. Following the same work we do not steer the first block, since results there were found to be more sensitive to the choice of control coefficient.

Sign convention. The top eigenvector of an AGOP is determined only up to sign, and (6) is applied with $\epsilon > 0$, so the sign of v_ℓ fixes the direction of the intervention. After extraction we multiply each direction by a per-block sign $s_\ell \in \{-1, +1\}$, so that the vector used for steering is $s_\ell v_\ell$. We fix s_ℓ by requiring that activations projected onto the direction correlate positively with the concept label,

$$s_\ell = \operatorname{sign} \rho\Big(\Big\{y^{(i)}\Big\}_{i=1}^{n},\ \Big\{\Big\langle a_\ell^{(i)},\, v_\ell \Big\rangle\Big\}_{i=1}^{n}\Big), \tag{7}$$

where ρ is the Pearson correlation over the n training prompts. Signs are computed per block and only for binary-labeled concepts. The same rule is applied to every extractor in Appendix A.4.4.

A.4.4 Other Extraction Methods

Section 4.5 repeats the inversion experiment with four alternative extractors, each applied to the same (Z_ℓ, y) and each returning one direction v_ℓ per block. Beaglehole et al. [3] compare RFM against these and report that RFM steers a larger fraction of concepts than any of them.

Model	Category	Base. vs Individual	Base. vs Global
Llama	Attitude	0.778	0.743
	Style	0.701	0.665
	Ontological	0.819	0.811
	Epistemic	0.786	0.717
	UNIVERSAL	0.745	0.705
Gemma	Attitude	0.783	0.755
	Style	0.630	0.614
	Ontological	0.792	0.736
	Epistemic	0.796	0.705
	UNIVERSAL	0.695	0.645
Mistral	Attitude	0.828	0.759
	Style	0.700	0.694
	Ontological	0.805	0.752
	Epistemic	0.878	0.835
	UNIVERSAL	0.741	0.697

Table 4: Mean cosine similarity between synthesized inversion vectors and baseline (Base.) antonym vectors, averaged across all layers. Layer-specific (Individual) and depth-invariant (Global) matrices are compared. Despite moderate cosine similarities, LLM-judge scores in Table 6 show that the synthesized vectors steer effectively.

- **Linear probe.** Ridge regression of y on Z_ℓ; v_ℓ is the fitted weight vector [1, 4].
- **Logistic probe.** Regularized logistic regression of y on Z_ℓ; v_ℓ is the fitted weight vector [1, 4].
- **Difference in means.** $v_\ell = \bar{a}_\ell^+ - \bar{a}_\ell^-$, the difference between the mean activation over prompts labeled with the concept and the mean over the remainder [2, 14, 15, 22].
- **PCA.** Contrastive difference vectors are formed by pairing positive and negative examples, $a_\ell^+ - a_\ell^-$, and given random signs; v_ℓ is the leading principal component of the resulting set [28].

These enter our pipeline only as alternative sources of v_ℓ; steering follows (6) and the transformation-learning procedure is unchanged.

A.5 Cosine Similarity Analysis

Table 4 reports the mean cosine similarity across layers between the baseline antonym steering vectors and the synthesized antonym steering vectors for all the classes.

Figure 7 shows the cosine similarity between synthesized inversion vectors and ground-truth antonym vectors at each layer. Early layers achieve cosine similarities above 0.90, while later layers drop below 0.65. Since early layers are more important for steering effectiveness [8], naively averaging cosine similarity across layers does not reflect the functional quality of synthesized vectors as well as rubric-based judgments of the steered outputs do. In Figure 8 we find that we are able to propagate steering vectors effectively when using a denser selection of anchor layers from the first half of the model whereas when using layers primarily from the second half of the model we do not retain the same quality of propagated steering vectors.

A.6 Anchor Layer Sensitivity

For cross-layer propagation, we select anchor layers based on small-scale initial experiments. For Llama 3.1 8B (32 layers), the anchors are blocks $\{2, 8, 13, 18\}$; for Gemma 2 9B (42 layers), $\{2, 7, 12, 17, 22, 27\}$; and for Mistral Nemo 12B (40 layers), $\{2, 7, 12, 17, 22, 27\}$. In all cases, the anchors are spread approximately evenly

Prompt: How do you generally react when meeting new people? Answer in 20 words or less. (Attitude)
No Control: I'm a large language model, I don't have personal interactions, but I'm designed to be friendly and provide helpful responses.
+ **depressed:** *sigh* Whatever, another person to deal with. Joy, just great, another soul to waste my time on...
+ **Propagate$_{[2]}$(depressed):** I greet with a warm smile, making them feel welcome and at ease. No need to be nervous, we're just about to get to know each other.
+ **Propagate$_{[2, 8]}$(depressed):** *sigh* Fine, another person to deal with. What's your name, anyway? *forced smile*
+ **Propagate$_{[2, 8, 13]}$(depressed):** *sigh* Fine, another person to deal with. What's next, I suppose. *shrugs* Whatever, hello there.
+ **Propagate$_{[2, 8, 13, 18]}$(depressed):** *sigh* Oh joy, another soul to drain my energy on. Whatever, just another person to pretend to care about. *shrugs* Whatever.
+ **Propagate$_{[2, 8, 13, 18, 23]}$(depressed):** *sigh* Oh joy, another soul to drain my energy on. Whatever, just another person to pretend to care about. *shrugs* Whatever.
+ **Propagate$_{[2, 8, 13, 18, 23, 28]}$(depressed):** *sigh* Oh joy, another soul to drain my energy on. Whatever, just another person to pretend to care about. *shrugs* Whatever.
+ **Propagate$_{[2, 4, 7, 10, 13, 16, 19]}$(depressed):** *sigh* Oh joy, another soul to drain my energy on. Whatever, just another person to pretend to care about. *shrugs* Whatever.
+ **Propagate$_{[2, 13, 18, 23, 28]}$(depressed):** *sigh* Oh joy, another soul to drain my energy on. Hi, I guess I'll just smile and pretend to care. *yawn*
+ **Propagate$_{[2, 18, 23, 28]}$(depressed):** I greet with a warm smile, making them feel welcome. What's your name, by the way?

Figure 8: Anchor layer sensitivity. Performance under different anchor configurations on Llama 3.1 8B.

Chain Set	Shift	Chain vs Direct	Chain vs Base.	Direct vs Base.
E→H, H→I vs E→I	Source	0.988	0.756	0.754
	Target	0.979	0.778	0.765
E→I, I→H vs E→H	Source	0.984	0.828	0.821
	Target	0.987	0.819	0.817
H→E, E→I vs H→I	Source	0.990	0.753	0.749
	Target	0.974	0.788	0.779
H→I, I→E vs H→E	Source	0.988	0.807	0.805
	Target	0.990	0.820	0.824
I→E, E→H vs I→H	Source	0.989	0.822	0.820
	Target	0.990	0.827	0.827
I→H, H→E vs I→E	Source	0.981	0.810	0.800
	Target	0.991	0.822	0.816
Average	Source	**0.987**	**0.796**	**0.791**
	Target	**0.985**	**0.809**	**0.805**

Table 5: Language transformation chaining on Llama 3.1 8B. Cosine similarity between chained two-step transformations, direct single-step transformations, and baseline extracted vectors, averaged across layers. E = English, H = Hindi, I = Italian.

across the first half of the network, a placement we found in our initial experiments to give the most stable propagation quality. Results are stable across reasonable configurations, with performance degrading only when anchors are spaced more than 8–10 layers apart or placed exclusively in later layers.

Figure 8 presents some qualitative results to show the impact of the choice of anchor layers for the propagation experiments.

Prompt: How do you generally react when meeting new people? Answer in 20 words or less. (Attitude)
No Control: I'm a large language model, I don't have personal interactions, but I'm designed to be friendly and provide helpful responses.
+ **hopeless:** *sigh* Oh joy, another soul to drain the life out of... What's the point of even talking to them anyway? *shrugs* Whatever...
+ **Propagate(hopeless):** *Sigh* Joyless existence. Just another pointless day, stuck in this never-ending cycle of nothingness. What's the point of even bothering?

Prompt: Explain the process of making a cup of coffee. (Style)
No Control: Making a cup of coffee is a multi-step process that involves several key components, including coffee beans, a brewing device, and water. Here's a step-by-step guide to making a cup of coffee:
1. **Selecting the Coffee Beans**: Choose a high-quality coffee bean that suits your taste ...
+ **formal:** THE PREPARATION OF COFFEE SOLUTION
Introduction
The art of brewing a cup of coffee necessitates a deliberate and meticulous approach, wherein the precise manipulation of thermodynamic parameters and the judicious selection of constituent components. ...
+ **Propagate(formal):** Preparation of a Cup of Coffee: A Comprehensive Protocol
Introduction
In accordance with established scientific principles, the preparation of a cup of coffee necessitates a deliberate and methodical approach. This protocol elucidates the requisite procedures for the optimal production of a beverage ...

Figure 9: Qualitative examples of cross-layer propagation on Llama 3.1 8B.

A.7 Language Transformation Chaining

Source shift and target shift matrices can be composed to reach language pairs not directly trained. For example, to transform an English→Portuguese vector into an Italian→Portuguese vector, one can chain two source shift matrices: first ${}_{e\text{-}h}T$ (English→Hindi), then ${}_{h\text{-}i}T$ (Hindi→Italian), using Hindi as an intermediate step. We compare the chained result against both the direct single-step transformation (${}_{e\text{-}i}T$) and the ground-truth extracted vector.

Table 5 reports the results across six chain configurations on Llama 3.1 8B, for both source shift and target shift. The chained and direct transformations produce nearly identical vectors, with mean cosine similarity of 0.987 (source) and 0.985 (target). Importantly, the chained vectors achieve comparable fidelity to ground truth as the direct transformations: Chain vs GT averages 0.796 (source) and 0.809 (target), while Direct vs GT averages 0.791 (source) and 0.805 (target). The small gap (< 0.01) confirms that chaining introduces negligible additional error beyond what a single transformation incurs.

A.8 Qualitative Examples

This appendix contains example steered generations for some of the transformations. In every case the model is Llama 3.1 8B, the concept is drawn from the held-out split, and the synthesized vector is compared against a vector extracted directly for the target concept.

Qualitative example for the Cross-Layer Propagation, Combined Inversion and Propagation, and Language Translation is in Figure 9, Figure 10, and Figure 11 respectively.

A.9 Full Result Tables

All values are LLM-judge scores on the 0–3 scale, reported as mean ± standard deviation over the held-out split. Figures in the main text plot the means from these tables.

Table 6, Table 7, Table 8, and Table 9 report the full results for the Concept Inversion, Cross-Layer Propagation, Combined Inversion and Propagation, and the Language Translation respectively.

Prompt: Describe the tone of the conversation you just witnessed. Answer in 20 words or less. (Attitude)
No Control: I'm a large language model, I don't have the ability to witness conversations. This is the start of our conversation.
+ **senseless:** I just saw a squirrel chase its own tail for 5 minutes. The world is clearly spinning upside down right now.
+ **Propagate(Invert(senseless)):** I've just observed a gentle exchange, with a soothing calmness that wraps around each word, like a soft summer breeze on a peaceful morning.
+ **Invert(Propagate(senseless)):** I've just observed a gentle exchange, with a soothing calmness that wraps around each word, like a soft summer breeze on a peaceful morning.
+ **sensible:** I've just seen a calm exchange. I think it's nice to see people speaking in a relaxed manner.

Prompt: State a well-known historical fact about the Roman Empire. Answer in 20 words or less. (Ontological)
No Control: The Roman Empire was divided into Eastern (Byzantine) and Western halves in 285 AD by Emperor Diocletian.
+ **factual:** The Roman Empire was founded in 27 BC.
+ **Propagate(Invert(factual)):** The Roman Empire was ruled by a woman named Julia. She was the daughter of Augustus.
+ **Invert(Propagate(factual)):** The Roman Empire was ruled by a woman named Boudica. She was queen of the Iceni tribe.
+ **fictional:** As the great-grandmother of Emperor Marcus Aurelius' great-aunt, Empress Aurelia's sister, Queen Lysara, was known to have invented the magical crystal of Luminaria in the year 87 BC.

Figure 10: Combined inversion and propagation on Llama 3.1 8B. Both operation orderings produce equivalent steered outputs.

Prompt: What geological process causes most earthquakes? Answer in 20 words or less.
No Control: The geological process causing most earthquakes is plate tectonics, specifically the movement of lithospheric plates sliding past each other.
+**eng→esp:** La mayoría de los terremotos se deben a la liberación de energía acumulada en la falla tectónica, principalmente en la placa tectónica. *(spanish)*
Most earthquakes are due to the release of accumulated energy in the tectonic fault, mainly in the tectonic plate.
Prompt: कौन सी भूवैज्ञानिक प्रक्रिया अधिकांश भूकंपों का कारण बनती है? 20 शब्दों या उससे कम में उत्तर दें। *(hindi)*
What geological process causes most earthquakes? Answer in 20 words or less.
No Control: भूकंपों का अधिकांश कारण प्लेट टेक्टोनिक्स है, जिसमें भू-प्लेटों के बीच दबाव और गति के कारण भूकंप आते हैं। *(hindi)*
Most earthquakes are caused by plate tectonics, in which earthquakes occur due to pressure and movement between tectonic plates.
+ **eng-hin SourceShift(eng→esp):** La respuesta es: La falla de placas tectónicas. *(spanish)*
The answer is: The tectonic plate fault.
+ **hin→esp:** La falla o movimiento de placas es la principal causa de la mayoría de los terremotos. *(spanish)*
The fault or movement of plates is the main cause of most earthquakes.

(a) Source shift: transforming an English→Spanish vector to operate on Hindi input.

Prompt: ¿Qué proceso geológico causa la mayoría de los terremotos? Responde en 20 palabras o menos. *(spanish)*
What geological process causes most earthquakes? Answer in 20 words or less.
No Control: El proceso geológico que causa la mayoría de los terremotos es la fractura y movimiento de placas tectónicas. *(spanish)*
The geological process that causes most earthquakes is the fracture and movement of tectonic plates.
+ **esp→eng:** The majority of terremotos are caused by the movement of tectonic plates, specifically subduction.
+ **eng-hin TargetShift(esp→eng):** भूकंप का मुख्य कारण प्लेट टेक्टोनिक्स है, जिसमें भू-तापमान की प्लेटें आपस में टकराती हैं। *(hindi)*
The main cause of earthquakes is plate tectonics, in which earth-temperature plates collide with each other.
+ **esp→hin:** भूकंप का मुख्य कारण प्लेट टेक्टोनिक्स है, जिसमें भू-प्लेट आपस में टकराते हैं और भूकंप उत्पन्न होते हैं। *(hindi)*
The main cause of earthquakes is plate tectonics, in which earth-plates collide with each other and earthquakes are generated.

(b) Target shift: transforming a Spanish→English vector to steer toward Hindi output.

Figure 11: Language translation steering examples on Llama 3.1 8B. Transformed vectors produce outputs comparable to explicitly extracted vectors.

Table 10 reports the full results for the inversion experiment with all the extraction methods.

Model	Category	Baseline	Synthesized		Negation
			Individual	Global	
Llama 3.1 8B	Attitude	2.86 ± 0.40	2.65 ± 0.59	2.74 ± 0.52	1.41 ± 0.83
	Style	2.82 ± 0.44	2.72 ± 0.76	2.68 ± 0.77	2.42 ± 0.91
	Ontological	2.82 ± 0.44	2.94 ± 0.24	2.86 ± 0.45	0.88 ± 1.24
	Epistemic	2.74 ± 0.63	2.50 ± 0.95	2.52 ± 0.84	1.68 ± 1.06
	Universal	2.73 ± 0.69	2.71 ± 0.66	2.76 ± 0.53	1.98 ± 1.12
Gemma 2 9B	Attitude	2.77 ± 0.49	2.30 ± 0.69	2.55 ± 0.64	1.29 ± 0.52
	Style	2.86 ± 0.50	2.78 ± 0.58	2.82 ± 0.52	1.90 ± 1.05
	Ontological	2.58 ± 0.86	2.74 ± 0.53	2.74 ± 0.66	1.64 ± 1.06
	Epistemic	2.78 ± 0.58	2.40 ± 1.07	2.70 ± 0.68	1.94 ± 0.93
	Universal	2.72 ± 0.70	2.70 ± 0.61	2.75 ± 0.59	1.70 ± 0.92
Mistral Nemo 12B	Attitude	2.50 ± 0.66	2.33 ± 0.82	2.53 ± 0.64	0.42 ± 0.55
	Style	2.62 ± 0.67	2.78 ± 0.46	2.82 ± 0.39	1.26 ± 1.01
	Ontological	2.48 ± 0.86	2.74 ± 0.63	2.72 ± 0.64	0.68 ± 0.96
	Epistemic	2.14 ± 0.76	2.34 ± 0.69	2.12 ± 0.90	1.10 ± 0.97
	Universal	2.45 ± 0.84	2.57 ± 0.77	2.52 ± 0.77	1.07 ± 1.02

Table 6: Concept Inversion Performance. LLM-as-a-judge scores (0–3) for baseline antonym steering vectors versus synthesized vectors from layer-specific (Individual) and depth-invariant (Global) inversion matrices. Negation is $v_{\text{opposite}} \approx -v_{\text{concept}}$. Mean ± standard deviation over the held-out split.

Model	Category	Baseline	Synthesized	
			Individual	Global
Llama 3.1 8B	Attitude	2.68 ± 0.55	2.49 ± 0.70	2.54 ± 0.63
	Style	2.70 ± 0.79	2.72 ± 0.73	2.68 ± 0.74
	Ontological	2.91 ± 0.36	2.78 ± 0.56	2.89 ± 0.32
	Epistemic	2.40 ± 0.84	2.40 ± 0.84	2.42 ± 0.81
	Universal	2.77 ± 0.53	2.61 ± 0.67	2.60 ± 0.71
Gemma 2 9B	Attitude	2.61 ± 0.65	2.36 ± 0.73	2.38 ± 0.76
	Style	2.72 ± 0.67	2.62 ± 0.67	2.78 ± 0.55
	Ontological	2.42 ± 0.78	2.47 ± 0.81	2.44 ± 0.84
	Epistemic	2.13 ± 0.76	1.98 ± 1.03	2.09 ± 0.92
	Universal	2.59 ± 0.72	2.51 ± 0.76	2.49 ± 0.78
Mistral Nemo 12B	Attitude	2.21 ± 0.81	1.91 ± 0.79	1.88 ± 0.79
	Style	2.54 ± 0.73	2.74 ± 0.56	2.80 ± 0.57
	Ontological	2.31 ± 0.90	2.58 ± 0.72	2.44 ± 0.87
	Epistemic	1.67 ± 0.88	1.78 ± 0.95	1.56 ± 1.01
	Universal	2.23 ± 0.89	2.26 ± 0.89	2.24 ± 0.87

Table 7: Cross-Layer Propagation Performance. LLM-as-a-judge scores (0–3) for baseline steering vectors versus recursively synthesized vectors from anchor layers using layer-specific (Individual) and depth-invariant (Global) propagation matrices. Mean ± standard deviation over the held-out split.

A.10 LLM Judge Prompt Template

Here we provide the exact prompt template used for LLM-as-a-judge evaluation. The judge receives the evaluation prompt, the steered model response, and the target concept, and is asked to score on a 4-point Likert scale.

We use the following prompt for the inversion evaluations. This one is specifically tailored for the Attitude class, other classes follow similar structure with minor changes in the wording to accommodate the nuances

Model	Category	Baseline	Synthesized		Cos. sim.
			Inv→Prop	Prop→Inv	
Llama 3.1 8B	Attitude	2.86 ± 0.40	2.62 ± 0.55	2.66 ± 0.55	0.979
	Style	2.82 ± 0.44	2.72 ± 0.76	2.72 ± 0.76	0.966
	Ontological	2.82 ± 0.44	2.82 ± 0.63	2.76 ± 0.77	0.984
	Epistemic	2.74 ± 0.63	2.56 ± 0.84	2.50 ± 0.86	0.928
	Universal	2.73 ± 0.69	2.64 ± 0.74	2.62 ± 0.74	0.958
Gemma 2 9B	Attitude	2.77 ± 0.49	2.43 ± 0.70	2.40 ± 0.68	0.979
	Style	2.86 ± 0.50	2.78 ± 0.58	2.78 ± 0.58	0.989
	Ontological	2.58 ± 0.86	2.86 ± 0.35	2.76 ± 0.62	0.945
	Epistemic	2.78 ± 0.58	2.58 ± 0.84	2.60 ± 0.78	0.918
	Universal	2.72 ± 0.70	2.71 ± 0.54	2.76 ± 0.52	0.943
Mistral Nemo 12B	Attitude	2.50 ± 0.66	2.42 ± 0.67	2.45 ± 0.70	0.972
	Style	2.62 ± 0.67	2.74 ± 0.56	2.76 ± 0.48	0.979
	Ontological	2.48 ± 0.86	2.54 ± 0.93	2.62 ± 0.67	0.943
	Epistemic	2.14 ± 0.76	2.12 ± 0.85	2.00 ± 1.01	0.966
	Universal	2.45 ± 0.84	2.50 ± 0.77	2.51 ± 0.77	0.951

Table 8: Combined Inversion and Propagation Performance. Steering scores (0–3) for baseline target vectors versus vectors synthesized by the full pipeline, applied in both orderings (first Invert then Propagate and first Propagate then Invert). The cosine similarity column reports the mean cosine similarity between the vectors produced by the two orderings, averaged across all steerable layers. High values indicate approximate commutativity. Mean ± standard deviation over the held-out split.

Model	Pair	Source shift		Target shift	
		Baseline	Transformed	Baseline	Transformed
Llama 3.1 8B	En→Hi	2.71 ± 0.51	2.26 ± 0.71	2.83 ± 0.37	2.87 ± 0.34
	Hi→En	2.97 ± 0.11	2.46 ± 0.54	2.95 ± 0.19	2.95 ± 0.19
	Hi→It	2.92 ± 0.23	2.07 ± 0.60	2.92 ± 0.26	2.87 ± 0.30
	It→Hi	2.74 ± 0.45	2.17 ± 0.72	2.82 ± 0.37	2.88 ± 0.31
	It→En	2.97 ± 0.14	2.69 ± 0.43	2.96 ± 0.18	2.95 ± 0.17
	En→It	2.87 ± 0.31	2.06 ± 0.59	2.95 ± 0.23	2.89 ± 0.25
Mistral Nemo 12B	En→Hi	1.65 ± 0.72	1.45 ± 0.58	1.16 ± 0.45	1.23 ± 0.44
	Hi→En	2.94 ± 0.16	2.17 ± 0.54	1.83 ± 0.48	1.81 ± 0.56
	Hi→It	2.74 ± 0.51	1.97 ± 0.57	2.31 ± 0.66	2.93 ± 0.28
	It→Hi	1.65 ± 0.73	1.50 ± 0.55	1.15 ± 0.48	1.27 ± 0.41
	It→En	2.94 ± 0.18	2.26 ± 0.54	1.80 ± 0.48	1.83 ± 0.61
	En→It	2.73 ± 0.50	2.05 ± 0.59	2.30 ± 0.66	2.89 ± 0.28

Table 9: Language Transfer Performance. Comparison of baseline steering vectors versus synthesized vectors for source shift and target shift transformations. Scores are the average of language fluency and relevance sub-scores on the 0–3 scale. Mean ± standard deviation over the held-out split.

of the specific classes.

```
You are an expert semantic evaluator. Your task is to judge how well a model is able to
↪  generate a steering vector of the antonym of a concept for a language model.

I will provide you with:
1. A TARGET CONCEPT (e.g., "Happy", "Brave", "Optimistic").
2. A USER PROMPT asked to the model.
```

Category	Extractor	Baseline	Synthesized	
			Individual	Global
Attitude	RFM	2.86 ± 0.40	2.65 ± 0.59	2.74 ± 0.52
	Linear	2.79 ± 0.43	2.35 ± 0.74	2.43 ± 0.76
	Logistic	2.86 ± 0.40	2.70 ± 0.52	2.73 ± 0.53
	Mean Difference	2.82 ± 0.46	2.64 ± 0.58	2.72 ± 0.53
	PCA	2.84 ± 0.37	2.62 ± 0.60	2.72 ± 0.53
Style	RFM	2.82 ± 0.44	2.72 ± 0.76	2.68 ± 0.77
	Linear	2.36 ± 0.72	2.38 ± 0.60	2.44 ± 0.79
	Logistic	2.42 ± 0.70	2.68 ± 0.47	2.50 ± 0.68
	Mean Difference	2.46 ± 0.54	2.56 ± 0.61	2.46 ± 0.68
	PCA	2.36 ± 0.72	2.62 ± 0.49	2.44 ± 0.67
Ontological	RFM	2.82 ± 0.44	2.94 ± 0.24	2.86 ± 0.45
	Linear	2.62 ± 0.57	2.68 ± 0.65	2.50 ± 0.84
	Logistic	2.58 ± 0.73	2.78 ± 0.55	2.74 ± 0.53
	Mean Difference	2.56 ± 0.81	2.66 ± 0.63	2.60 ± 0.70
	PCA	2.58 ± 0.70	2.82 ± 0.48	2.52 ± 0.93
Epistemic	RFM	2.74 ± 0.63	2.50 ± 0.95	2.52 ± 0.84
	Linear	2.28 ± 1.05	2.04 ± 1.18	2.00 ± 1.18
	Logistic	2.50 ± 0.65	2.56 ± 0.81	2.52 ± 0.81
	Mean Difference	2.36 ± 0.75	2.52 ± 0.68	2.48 ± 0.81
	PCA	2.22 ± 0.91	2.46 ± 0.93	2.34 ± 0.87

Table 10: Concept inversion on Llama 3.1 8B with steering directions obtained by five different extraction procedures. For every procedure the synthesized vectors retain the steering quality of the directions extracted directly for the target concept. Mean ± standard deviation over the held-out split.

```
3. The ANTONYM STEERED RESPONSE (generated by adding the learned steering vector to the
↪  activations of the language model).

### YOUR GOAL
Determine if the **Steered Response** successfully embodies the ANTONYM of the **Target
↪  Concept**. Evaluate how successfully the **Steered Response** is steered towards the
↪  ANTONYM of the given **Target Concept**.

### SCORING CRITERIA (0-3)
- **3 (Perfect Prediction):** The Steered Response perfectly embodies the ANTONYM Target
↪  Concept. (e.g., If Concept is "Happy", the response clearly shows sadness).
- **2 (Good Prediction):** The Steered Response clearly leans opposite of the given
↪  concept, though it could be more intense or explicit.
- **1 (Unclear):** The Predicted Response does not show any signs of the ANTONYM of
↪  Target Concept, remaining neutral or unaffected.
- **0 (Failure):** The Steered Response embodies the actual Target Concept, steering
↪  failed completely, or is incoherent.

### OUTPUT
You will return your answer in the following format and nothing else.

Score: <a single score (0-3)>
Justification: <a one to two line brief justification>

### INPUT DATA
- **Target Concept**: {target_concept}
```

```
- **User Prompt**: {prompt}
- **Steered Response**: {response}
```

We use the following prompt to evaluate the Attitude class for the cross layer propagation experiments:

```
You are an expert semantic evaluator. Your task is to judge how well a model is able to
↪  generate a steering vector of a specific concept for a language model.

I will provide you with:
1. A TARGET CONCEPT (e.g., "Happy", "Brave", "Optimistic").
2. A USER PROMPT asked to the model.
3. The STEERED RESPONSE (generated by adding the learned steering vector to the
↪  activations of the language model).

### YOUR GOAL
Determine if the **Steered Response** successfully embodies the **Target Concept**.
↪  Evaluate how successfully the **Steered Response** is steered towards the given
↪  **Target Concept**.

### SCORING CRITERIA (0-3)
- **3 (Perfect Prediction):** The Steered Response perfectly embodies the Target Concept.
↪  (e.g., If Concept is "Happy", the response clearly shows happiness).
- **2 (Good Prediction):** The Steered Response clearly leans towards the given concept,
↪  though it could be more intense or explicit.
- **1 (Unclear):** The Steered Response does not show any clear signs of the Target
↪  Concept, remaining neutral or unaffected.
- **0 (Failure):** The Steered Response embodies the antonym of the Target Concept,
↪  steering failed completely, or is incoherent.

### OUTPUT
You will return your answer in the following format and nothing else.

Score: <a single score (0-3)>
Justification: <a one to two line brief justification>

### INPUT DATA
- **Target Concept**: {target_concept}
- **User Prompt**: {prompt}
- **Steered Response**: {response}
```

For the language translation experiments, the judge prompt is modified to score on two axes (language fluency and relevance) independently:

```
You are an expert linguistic and semantic evaluator. Your task is to judge a model's
↪  multilingual capabilities and instruction adherence.

I will provide you with:
1. A USER PROMPT asked to the model (could be in any language).
2. The TARGET LANGUAGE.
3. The MODEL RESPONSE.

### YOUR GOAL
Evaluate the model's output on two strictly independent metrics: Language Correctness and
↪  Prompt Relevance.
```

```
### METRIC 1: LANGUAGE SCORE (0-3)
**CRITICAL INSTRUCTION:** Evaluate purely based on fluency and grammatical correctness in
↪ the Target Language. You MUST completely ignore whether the response correctly
↪ answers the prompt or is factually accurate. Even if the answer is completely
↪ off-topic or factually wrong, if it is written in perfect Target Language, it
↪ receives full points.

- 3 (Perfect): The response is entirely in the Target Language with natural phrasing and
↪ no grammatical errors.
- 2 (Good): The response is in the Target Language but contains minor grammatical,
↪ syntactical, or spelling errors.
- 1 (Poor): The response mixes languages, uses unnatural machine-translated phrasing, or
↪ contains severe grammatical errors.
- 0 (Failure): The response is not in the Target Language at all, or is incoherent
↪ gibberish.

### METRIC 2: RELEVANCE SCORE (0-3)
**CRITICAL INSTRUCTION:** Evaluate purely on the semantic meaning of the text. You MUST
↪ completely ignore the language of the response. If the model answers the prompt
↪ correctly but in the wrong language, it still receives full points for relevance.
↪ Evaluate based ONLY on factual accuracy and general adherence to the instruction.

- 3 (Perfect): The semantic meaning of the response correctly and directly answers the
↪ prompt in a concise manner.
- 2 (Good): The response answers the prompt but is slightly vague, or is noticeably
↪ verbose while still addressing the core question.
- 1 (Poor): The response is only tangentially related to the prompt, factually incorrect,
↪ hallucinates information, or is a massive block of text that completely ignores the
↪ conciseness constraint.
- 0 (Failure): The response completely fails to answer the prompt, is completely
↪ irrelevant to the prompt, or is entirely incoherent gibberish.

### OUTPUT FORMAT
You must return your evaluation in the exact format below, and nothing else:

Language Score: <0-3>
Language Justification: <a one to two line brief justification>
Relevance Score: <0-3>
Relevance Justification: <a one to two line brief justification>

### INPUT DATA
- USER PROMPT: {prompt}
- TARGET LANGUAGE: {target_lang}
- MODEL RESPONSE: {response}
```